

Zine

Haskell is the glue, so that you can think about the parts in isolation. Category theory style

This resolves a nominal into a specific noun

```
resolve :: Nominal -> Noun
```

These are all the thirty-seven relations in the Universal Dependencies grammar

```
data Relation = Nsubj
              | Obj
              | Iobj
              deriving (Eq, Ord, Enum)

newtype Dependency source target = Dependency (source -> ([Relation], target))
```

Because Dependency is generic on any source or target, coreference can be handled separately on a PoS layer I bet

```
--instance Category Dependency where
--  id = Dependency $ \source -> ([], source)
--  (Dependency a) . (Dependency b) = Dependency $ \source ->
--    let (relations, intermediate) = b source
--        (relations', target) = a intermediate
--    in (relations ++ relations', target)
```

Dependency parsing happens sentence-by-sentence

```
type Syntax = [Sentence]
```

UDPipe (maybe see <https://github.com/fpco/inline-c>), lazily

```
dependencyParse :: Text -> Syntax
```

Functor from the tree-category of Dependency to a meaningful category. See below A DEPENDENCY TREE TOPOS – use Prolog

```
understand :: Syntax -> Maybe Semantics
explain :: Semantics -> Syntax
```

Maxima here, which needs a ECL FFI or general Lisp IPC

```
simplify :: Math -> Math -- Maxima
```

Yi, The keybindings part

```
interpret :: Input -> Command
```

Yi, stuff from EditorM for example

```
execute :: Command -> State -> State
```

Custom, using typst-layout, which needs rust FFI or some (??) IPC

```
view :: State -> DisplayList
```

WebRender ofc, which needs rust FFI (see <https://github.com/harpocrates/inline-rust>) or general Rust IPC

```
webrender :: DisplayList -> GL
```

```
createFrame :: IO GLContext
```

```
read :: IO Input
```

The main interaction loop

```
main :: IO ()
main = do
  frame <- createFrame
  let interactionLoop state = do
    display ((webrender . view) state) frame
    input <- read
    let state' = (execute . interpret) input state
    interactionLoop state'
  interactionLoop initialState
```

Profane definitions of types

```
newtype GL = GL { display :: GLContext -> IO () }
```

Haskell default and glue:

- Prolog (NLP Topos FOL). Maybe this can be in native Haskell or scheme
- Lisp (CAS) Better interop with scheme
- Rust (rendering)
- C++ (text tagging, Firefox IPDL) Can be done with udpipes

lol maybe idris instead of haskell -> scheme -> rust marwood or other scheme

implementation for display list, udpipes -> prolog with call/cc -> lisp direct interop with scheme lists but embedded via ecl2

if they're all separate, you could get a prolog repl, an ECL repl, and a GHCi repl

maybe prolog translates directly into lispy things

haskell -> prolog -> lisp

1. Philosophy and Sheaf Theory

Need to think about the pushout of rooted words, which would connect subjects to objects via some actual relation. Because of the categorical structure of Dependency, only the dependency paths between two different tokens matters. Pushouts can be related to specific morphisms, and will probably be through things like verbs

If I was to use sheaf theory here, then I've broken the problem into:

- Local semantics
- gluing, including coreference probably
- Global semantics

Globally, I want to extract stuff like

```
newtype Epistemology = Epistemology (Set Fact -> Knowledge)
```

What is true about both the local semantics and the global semantics? Both subcategories of Hask, likely, as they both need to be representable internally by the computer ultimately.

So what is Epistemology? It's a subcategory with only two objects and it goes unidirectionally. pushouts are what two epistemic systems have in common and pullbacks are the dual

what about hegel

```
dialectic :: (Thesis, Thesis) -> Maybe Thesis
```

metaphysics: what is? epistemology: what is knowledge? ethics: what is just/right/needful?
probably can't constrain this too much – maybe it's literally just collections of morphisms between two Hask objects

```
type Philosophy a b = [a -> b]
```

ok now we might be getting somewhere, as I can also make morphisms from local semantics via pushouts through a tree root in the dependency parse. I don't necessarily have to ask from whence nouns came from (this is restricted to humans), just the relationships defined between them.

2. A Dependency Tree Topos

Topos of local truth constructed in just one sentence. First add necessary pushouts of morphisms through relation words to construct just noun phrases as objects in the category. Objects are nominals that are related to others

Maybe– Terminal object: “this”/unrestricted PRON Binary products: different sentences, indicates that two independent clauses can be broken up Equalizer: CCONJ/union
Exponential: linguistic recursion/which Omega: “itself” all nouns Sub object classifier: X–copula–>“itself”

Existential quantification would use “this”, whereas the default is just universal. i.e. Red is bright vs this red is bright.

Now that I have a sheaf topos, I would like to be able to glue it together into a general understanding. The glue will inevitably have a nonzero cohomology in which case we will fail to synthesize a global understanding. But say we have a global understanding; then we can understand how different parts fit together by alternating between natural language representation and the categorical.

Once I have a local topos, I can attempt gluing using some kind of ACL2 combo or H-M unification, or both. I will need to figure this out later, and the exact system by which the local, overfit, understanding gets generalized to the global will determine the cohomology.

Maybe have a two-level sheaf attempting to glue sentences together in one argument, using some natural deduction and coreference (H-M unification there?) in the glue between sentences which are part of a topos. Then, can try a different kind of glue to see how two arguments fit together, and which way the functors go between them. Maybe the higher level isn't even a sheaf and just a topology defined on it.

3. Category theory

```
class ConstrainedCategory (k :: Type -> Type -> Type) where
  type Object k (a :: Type) :: Constraint

  id :: Object k a => k a a

  (.) ::
    (Object k a, Object k b, Object k c) =>
    k b c -> k a b -> k a c

infixr 9 .
```

```
instance ConstrainedCategory (->) where
  type Object (->) a = ()
```

```
id x = x
```

```
(g . f) x = g (f x)
```

D. Referenced packages

```
{-# LANGUAGE ConstraintKinds #-}  
{-# LANGUAGE FlexibleInstances #-}  
{-# LANGUAGE KindSignatures #-}  
{-# LANGUAGE TypeFamilies #-}  
{-# LANGUAGE TypeOperators #-}
```

```
import Data.Kind (Constraint, Type)  
import Prelude hiding (id, (.), read)
```

Placeholder implementations and types

```
resolve = undefined  
dependencyParse = undefined  
understand = undefined  
explain = undefined  
simplify = undefined  
interpret = undefined  
execute = undefined  
view = undefined  
webrenderer = undefined  
createFrame = undefined  
read = undefined  
initialState = undefined  
dialectic = undefined
```

```
data Nominal = Nominal  
data Noun = Noun  
data Sentence = Sentence  
data Text = Text  
data Semantics = Semantics  
data Math = Math  
data Input = Input  
data Command = Command  
data State = State  
data DisplayList = DisplayList  
data GLContext = GLContext  
data Set a = Set  
data Fact = Fact  
data Knowledge = Knowledge  
data Thesis = Thesis
```